

Nanoleaf USB Lightstrip Communication Protocol

[Introduction](#)

[USB information](#)

[Offline Mode](#)

[Nanoleaf PC Screen Mirror Light-strip](#)

[Nanoleaf Pegboard Desk Dock](#)

[Protocol](#)

[List of Commands](#)

[Set RGB channels on Light-strip LED zones](#)

[Get Length of Light-strip](#)

[Get Device On Off State](#)

[Set Device On Off State](#)

[Get Device Brightness](#)

[Set Device Brightness](#)

[Get Device Firmware Version](#)

[Get Device Model Number](#)

[Button Press Event](#)

Introduction

This document describes the PC Screen Mirror LS protocol over USB. The PC Screen Mirror LS when connected to a PC or mac, exposes itself as a HID device.



This protocol applies to both, the Nanoleaf PC Screen Mirror Light Strip and the Nanoleaf Pegboard Desk Dock.

USB information

Product Name	Base Model Number	Sub Model Number	Full Model Number	PID (Hex)	VID (Hex)
Nanoleaf Pegboard Desk Dock	NL82	K1	NL82K1	0x8201	0x37FA
PC Screen Mirror LS	NL82	K2	NL82K2	0x8202	0x37FA

Offline Mode

If no command / message is recd from a connected PC by the device, the device enters Offline Mode.

In offline mode, the device will continue to display the last displayed state on the LEDs until a user interacts with the device using the buttons.

The button behaviour in offline mode is different and as follows:

Nanoleaf PC Screen Mirror Light-strip

Gesture / Button	Power	Scene
Single Press	Toggle On Off state	Cycle between default colours and scenes
Double Press	Toggle On Off state	Cycle between default colours and scenes
Long Press	Toggle On Off state	Cycle between default colours and scenes

Nanoleaf Pegboard Desk Dock

Gesture / Button	Button
Single Press	Cycle through default scenes
Double Press	Cycle through default solid colours
Long Press	Toggle On Off state

Both the Nanoleaf PC Screen Mirror Light-Strip and the Nanoleaf Pegboard Desk Dock come with default colours and default scenes preloaded on the device. These scenes cannot be removed or modified.

Protocol

The communication protocol is TLV based. The first byte of the message indicates the **type** of the message and the next two bytes indicate the **length** of the message encoded as **Big-Endian** followed by the payload of the message.

Every message sent to the controller will be responded with a response message encoded as a TLV in the same format as the message.

The type of the message will be `0x80 + message type`

List of Commands

Set RGB channels on Light-strip LED zones

Request

Command Type	Length	Value
0x02	<variable, max = 255 * 4 bytes>	RGB values for each LED zone of the Lightstrip, concatenated one after the other. The length of the Lightstrip in number of zones can be obtained using the length command. Each RGB tuple is 3B in length, 1B for R, G and B 1 RGB_0 RGB_1 RGB_2 RGB_3 ... RGB_N  The length of this payload must be a multiple of 3B.  The device's on-off state must be set to On and brightness must be a value greater than 0

Response

Response Type	Length	Value
0x82	1	1B Error Code: 0: Success

		1: Failure
--	--	------------

Get Length of Light-strip

Request

Command Type	Length	Value
0x03	0	N/A

Response

Response Type	Length	Value
0x83	2	1B Error Code 1B Light-strip length <u>Error Code</u> 0: Success 1: Failure

Get Device On Off State

Request

Command Type	Length	Value
0x06	0	N/A

Response

Response Type	Length	Value
0x86	2	1B Error Code 1B on-off state <u>Error Code</u> 0: Success 1: Failure

		<u>On Off State</u> 0x00 - Device is Off 0x01 - Device is On
--	--	--

Set Device On Off State

Request

Command Type	Length	Value
0x07	1	<u>On Off State</u> 0x00 - Device is Off 0x01 - Device is On

Response

Response Type	Length	Value
0x87	1	<u>Error Code</u> 0: Success 1: Failure

Get Device Brightness

Request

Command Type	Length	Value
0x08	0	N/A

Response

Response Type	Length	Value
0x88	2	1B Error Code 1B brightness <u>Error Code</u> 0: Success 1: Failure

		<u>Brightness</u> [0, 255]
--	--	-------------------------------

Set Device Brightness

 This will also modify the brightness of the device in [offline mode](#)

Request

Command Type	Length	Value
0x09	1	<u>Brightness</u> [0, 255]

Response

Response Type	Length	Value
0x89	1	<u>Error Code</u> 0: Success 1: Failure

Get Device Firmware Version

Request

Command Type	Length	Value
0x0A	0	N/A

Response

Response Type	Length	Value
0x8A	2	1B Error Code 1B firmware version <u>Error Code</u> 0: Success 1: Failure

		<u>Firmware Version</u> The 4MSB is the major version number, the 4LSB is the minor version number
--	--	---

Get Device Model Number

Request

Command Type	Length	Value
0x0C	0	N/A

Response

Response Type	Length	Value
0x8C	7	1B Error Code 6B model number <u>Error Code</u> 0: Success 1: Failure <u>Model Number</u> 6B ASCII characters representing the model number of the device, ex: NL82K2

Button Press Event

 The Button Press is an unsolicited message sent from the device to the PC.

Event Type	Length	Value
0x85	<variable, 2 * num_buttons>	The number of buttons on the device depends on the model.

For PC Screen Mirror Lightstrip, the number of buttons is two:

Id 1. Power Button

Id 2. Scene Button

For Pegboard Desk Dock, the number of buttons is just one, which is unmarked.

the event is encoded as a concatenated `button_id | button_gesture` tuple for each button.

Button Gesture Enum:

NONE = 0

SINGLE PRESS = 1

DOUBLE PRESS = 2

LONG PRES = 3

Ex: if the power button is double pressed on the PC Screen Mirror Lightstrip:

`0x01 0x01 0x02 0x00`

Ex: If the single button is double pressed on the Pegboard Desk Dock:

`0x01 0x02`